

Tema 2 | Análisis de Léxico

2.1 Descripción funcional.

2.2 Conceptos de token, lexema y patrón.

2.3 Fundamentos: Álgebra de lenguajes.

2.3.1 Alfabeto y lenguaje.

2.3.2 Operaciones con lenguajes.

2.3.3 Especificación de tokens y patrones.

2.3.4 Propiedades y notación taquigráfica de las expresiones regulares.

2.3.5 Autómata Finito no Determinista (AFN) asociado a una expresión regular.

2.3.6 AFN reconocedor de patrones.

2.3.7 Autómata Finito Determinista (AFD).

2.3.8 Gramática abstracta y tabla de tokens.

2.4 Tratamiento de errores.

2.4.1 Estrategias de recuperación.

2.4.2 Reparación de errores.

2.5 LEX (generador de analizadores de léxico).

2.5.1 Introducción.

2.5.2 Especificación LEX.

2.5.3 Consideraciones finales.

2.5.4 Ejemplos LEX.

Bibliografía básica

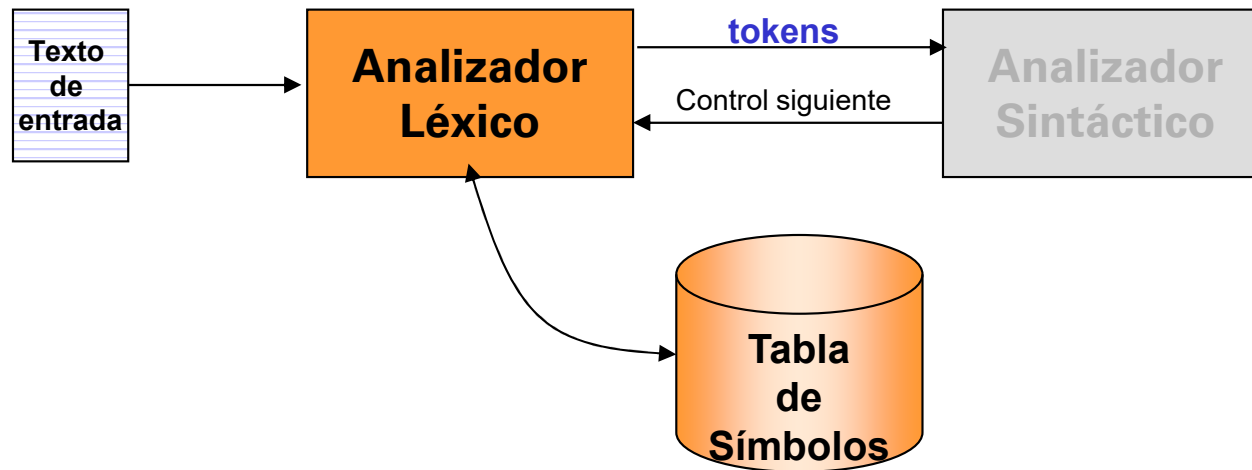
- [Aho90] Alfred V. Aho, Ravi Sethi, Jeffrey D. Ullman
Compiladores. Principios, técnicas y herramientas. Addison-Wesley Iberoamericana 1990.
- [Broo93] J.G. Brookshear
Teoría de la Computación. Lenguajes formales, autómatas y complejidad. Addison-Wesley Iberoamericana 1993

- [Levi92] J.R. Levine, T. Manson, D. Brown
Lex & Yacc
O'Reilly & Associates, Inc. 1992.
- [Benn90] J.P. Bennet
Introduction to compiling techniques: A first course using ANSI C, LEX y YACC
McGraw-Hill. 1990.

26/09/2026

2.1 Descripción funcional

Lee, carácter a carácter, del documento de entrada (texto fuente) y genera una secuencia de patrones léxicos denominados **tokens** y, en su caso, asocia **atributos** a los tokens.



Aportaciones del analizador de léxico

- Simplificar el diseño del **analizador sintáctico**, confiriéndole una mayor eficacia.
- Portabilidad del traductor.

2.2 Conceptos de token, lexema y patrón

- **Token:** Conjunto de secuencias de caracteres con la misma misión **sintáctica**.
- **Lexema:** Secuencia de caracteres que forman un **token**.
- **Patrón:** Regla o reglas que describen a los lexemas asociados a un **token**.

Ejemplo 2.1: Dada la especificación de este lenguaje.

```
P → S | P S
S → repeat exp S;
   | while exp S;
   | if exp S;
   | id = exp;
exp → exp * exp
    | exp / exp
    | exp + exp
    | + exp
    | * exp
    | (exp)
    | constante
    | id
```



Inicialmente, tomamos los siguientes lexemas: **repeat**, **while**, **if**.
¿qué tienen en común?... Forman parte de una sentencia **S** y van seguidos de “**exp S;**”

**COINCIDEN EN TODO LO QUE LE PRECEDE Y SUCEDE
EN UN TEXTO**

Es decir, es posible agrupar estos tres lexemas en otro de mayor abstracción para constituir un **token**.

Consecuencia: Reducimos tres reglas gramaticales en una sola:

$$S \rightarrow \text{CONDICLO } \text{exp } S ;$$
$$| \text{ id } = \text{exp } ;$$

Donde, **CONDICLO** es un token definido por el siguiente patrón:

("repeat" | "while" | "if")

2.3 Fundamentos: Algebra de lenguajes

2.3.1 Alfabeto y lenguaje

Alfabeto: Conjunto de símbolos. $A = a, b, \dots, z, A, B, \dots, Z$

Lenguaje: Conjunto de secuencias de símbolos de un alfabeto. A un alfabeto **A** le corresponde un *lenguaje elemental* que es **A**.

$$L = a, b, \dots, z, A, B, \dots, Z$$

Formas de especificar un lenguaje

Por *Extensión*: $L = \{a, aa, aba, abbaa, \dots\}$

Por *Comprensión*: **GRAMÁTICA**

2.3.2 Operaciones con lenguajes

- **Unión:** $L_1 \cup L_2 = \{x/x \in L_1 \text{ o } x \in L_2\}$
- **Intersección:** $L_1 \cap L_2 = \{x/x \in L_1 \text{ y } x \in L_2\}$
- **Concatenación:** $L_1 L_2 = \{xz/x \in L_1 \text{ y } z \in L_2\}$, $L^0 = \{\lambda\}$
- **Clausura:** $L^* = \bigcup_{i=0}^{\infty} L^i$
- **Semigrupo:** $L^+ = \bigcup_{i=1}^{\infty} L^i$

2.3.3 Especificación de tokens y patrones

Token	Código	Atributos	Patrón/Expresión regular
CONDCICLO	257	0: repeat 1: while 2: if	("repeat" "while" "if")

Expresiones regulares

Dado un alfabeto Σ las reglas que definen expresiones regulares de Σ son:

- λ es una expresión regular que representa la secuencia vacía: $\{\lambda\}$
- Si $a \in \Sigma$ entonces $\{a\}$ es una expresión regular
- Sea r y s dos expresiones regulares representadas por los lenguajes $L(r)$ y $L(s)$. Se cumple:
 - a) $(r) \mid (s)$ es una expresión regular representada por $L(r) \cup L(s)$
 - b) $(r) (s)$ es una expresión regular representada por $L(r) L(s)$
 - c) $(r)^*$ es una expresión regular representada por $L(r)^*$
 - d) (r) es una expresión regular representada por $L(r)$

2.3.4 Propiedades y notación taquigráfica de las expresiones regulares

Con las definiciones anteriores se cumplen las siguientes propiedades:

- Conmutativa: $r|s = s|r$.
- Asociativa: $r|(s|t) = (r|s)|t$.
- Asociativa sobre la concatenación: $(rs)t = r(st)$.
- Distributiva de la concatenación y unión: $r(s|t) = rs|rt$ y $(r|t)s = rs|ts$.
- Elemento neutro: $\lambda r = r$ y $r\lambda = r$, siendo λ el elemento neutro y también se puede expresar $r^* = (r|\lambda)^*$ y $r^{**} = r^*$.

También se define la siguiente notación taquigráfica para la representación de expresiones regulares:

1. Uno o más veces: operador $+$, $(r^* = r^+|\lambda)$.
2. Cero o una vez: operador $?$.
3. Cero o más veces: operador $*$.
4. Una forma cómoda de definir *clases de caracteres* es de la siguiente forma:

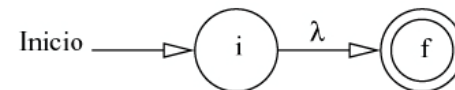
$$a|b|c|\dots|z = [a-z]$$

2.3.5 Autómata Finito no Determinista (AFN) asociado a una expresión regular

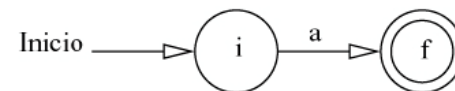
Teorema: A toda expresión regular le corresponde un AFN (demostración en [Broo93]).

La **Construcción de Thompson** permite obtener el AFN asociado a una expresión regular dada. Este algoritmo, a partir de las siguientes expresiones regulares, asocia los siguientes AFNs:

1. Para la secuencia vacía λ



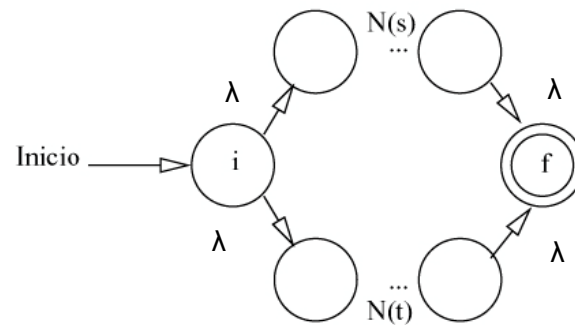
2. Para la secuencia $a \in \Sigma$



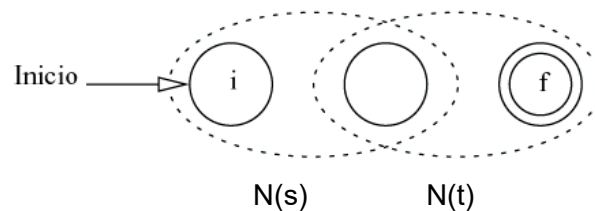
2.3.5 Autómata Finito no Determinista (AFN) asociado a una expresión regular

3. Sean s y t expresiones regulares y $N(s)$ y $N(t)$ los AFNs correspondientes, tenemos:

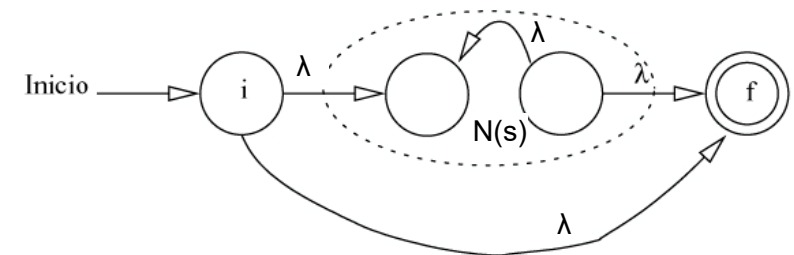
(a) Para $s|t$



(b) Para st



(c) Para expresiones s^* se construye el AFN $N(s^*)$

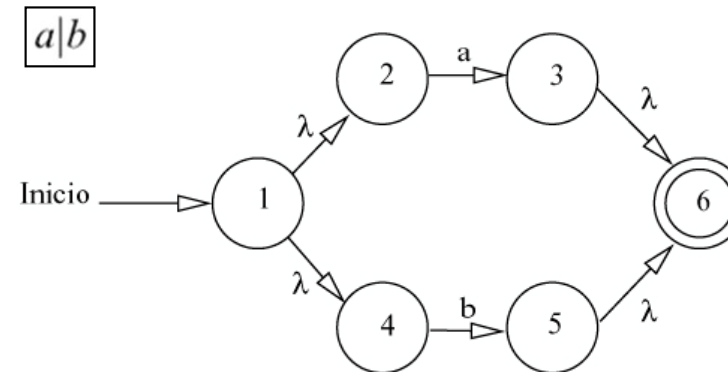
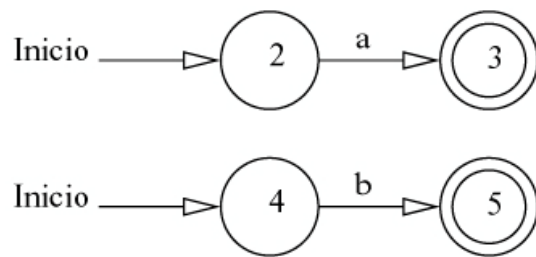


2.3.5 Autómata Finito no Determinista (AFN) asociado a una expresión regular

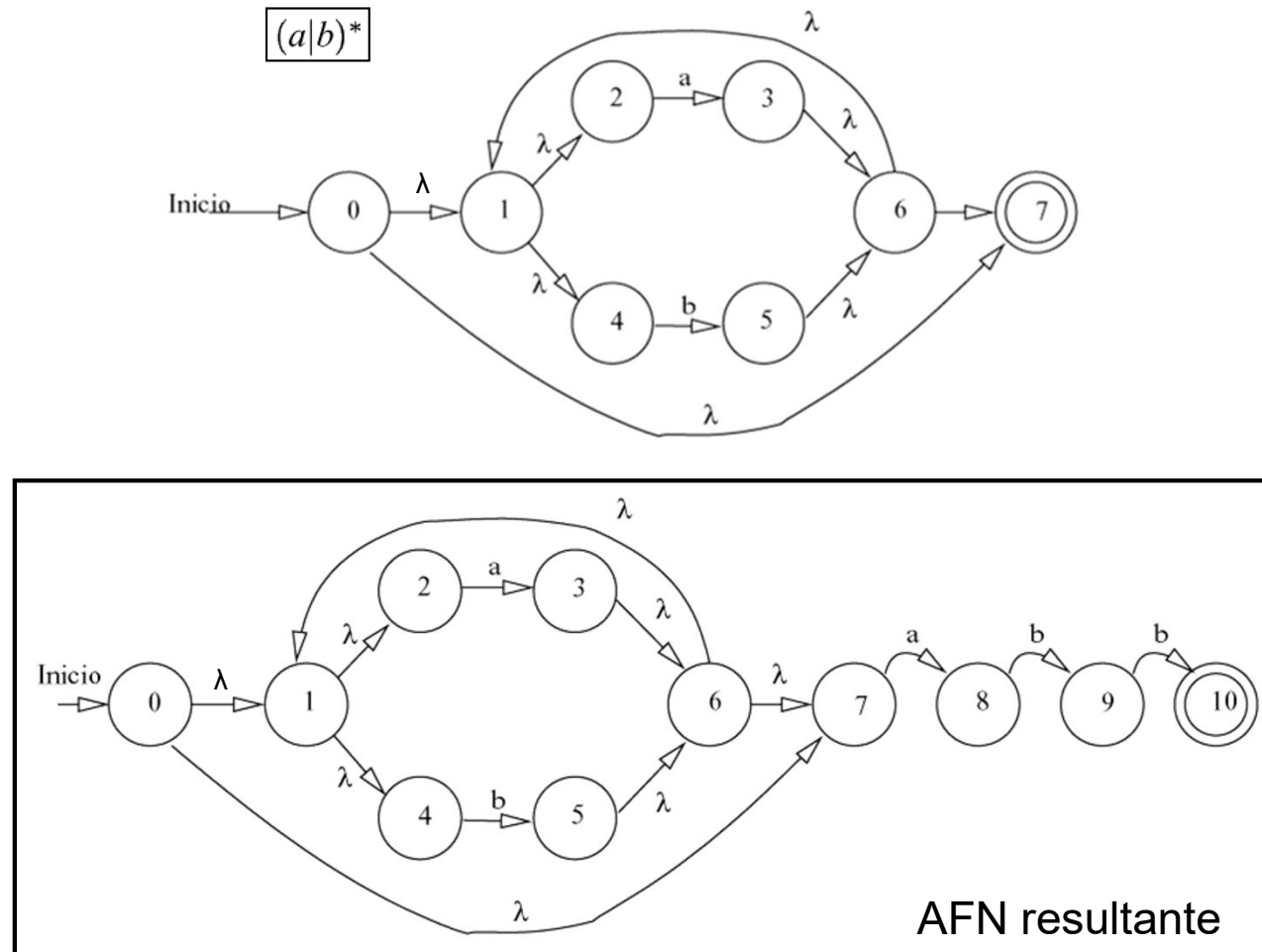
Supongamos la siguiente expresión regular

$$r = (a|b)^*abb$$

La forma de construir el AFN es por pasos. En primer lugar el de los símbolos a y b por separado:

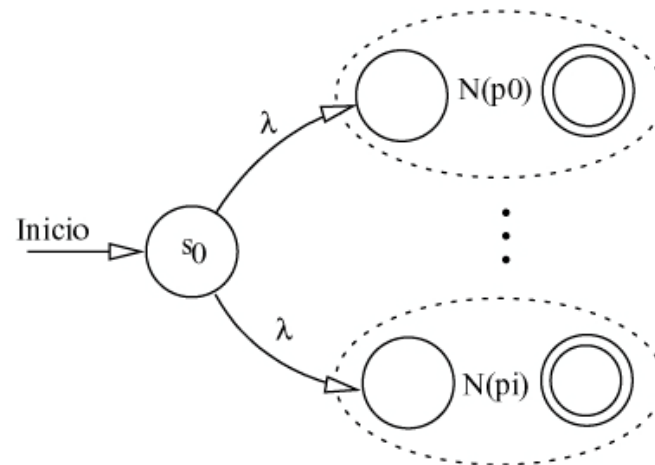


2.3.5 Autómata Finito no Determinista (AFN) asociado a una expresión regular



2.3.6 AFN reconocedor de patrones

Sean $p_0 \mid p_1 \mid p_2 \mid \dots \mid p_i$ los patrones que se deben reconocer. Se le asocia a cada patrón un AFN, añadiendo para cada patrón una **λ -transición** desde un estado inicial.



2.3.7 Autómata Finito Determinista (AFD)

Existe un teorema en Teoría de Autómatas por el que a todo AFN se le puede asociar un Autómata Finito Determinista (AFD) equivalente.

Un AFD es un caso especial de AFN en el que se cumplen las siguientes condiciones:

1. Ningún estado tiene **λ -transiciones**.
2. Para cada estado **s** y cada símbolo de entrada **a**, hay a lo sumo una arista etiquetada **a** que sale de **s**.

Para la conversión emplearemos tres funciones:

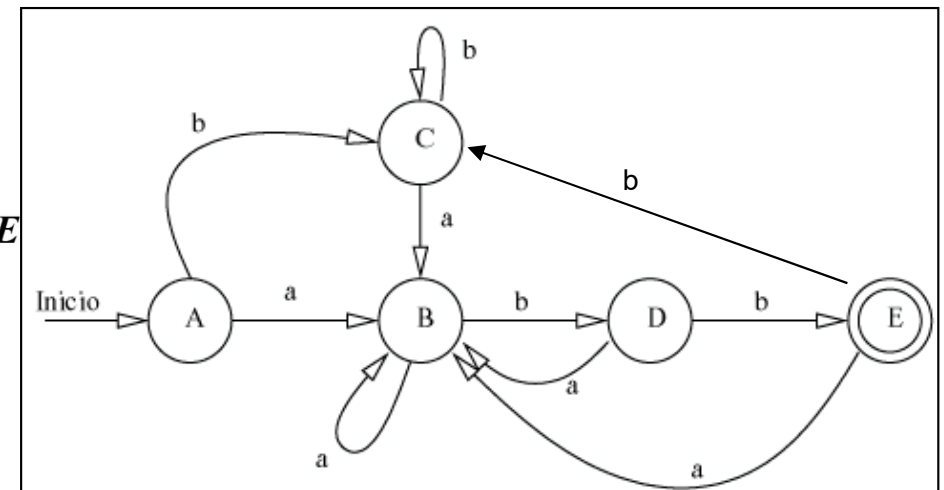
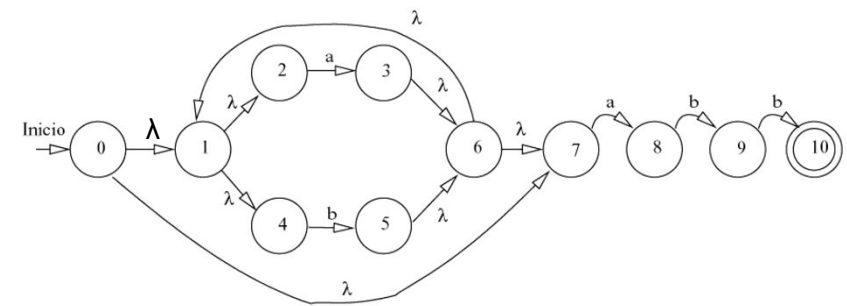
- **λ -transición(s)**: Conjunto de estados del AFN alcanzables desde el estado **s** únicamente con λ -transiciones.
- **λ -clausura(T)**: Conjunto de estados del AFN alcanzables desde algún estado **s** en **T** únicamente con λ -transiciones (siendo **T** un conjunto de estados).
- **Move(T,a)**: Conjunto de estados del AFN hacia los cuales hay una transición con el símbolo de entrada **a** desde algún estado **s** en **T**.

Si un estado **e** del AFD contiene algún estado final del AFN, entonces **e** es un estado final del AFD

2.3.7 Autómata Finito Determinista (AFD)

Ejemplo 2.2: Construcción del AFD equivalente a partir del AFN anterior. El alfabeto de entrada es {a,b}.

λ -clausura(0) = {0,1,2,4,7} = **A**
 λ -clausura(Move(A,a)) = λ -clausura({3,8}) = {1,2,3,4,6,7,8} = **B**
 Trans(A,a) = **B**
 λ -clausura(Move(A,b)) = λ -clausura({5}) = {1,2,4,5,6,7} = **C**
 Trans(A,b) = **C**
 λ -clausura(Move(B,a)) = λ -clausura({3,8}) = **B**
 Trans(B,a) = **B**
 λ -clausura(Move(B,b)) = λ -clausura({5,9}) = {1,2,4,5,6,7,9} = **D**
 Trans(B,b) = **D**
 λ -clausura(Move(C,a)) = λ -clausura({3,8}) = **B**
 Trans(C,a) = **B**
 λ -clausura(Move(C,b)) = λ -clausura({5}) = **C**
 Trans(C,b) = **C**
 λ -clausura(Move(D,a)) = λ -clausura({3,8}) = **B**
 Trans(D,a) = **B**
 λ -clausura(Move(D,b)) = λ -clausura({5,10}) = {1,2,4,5,6,7,10} = **E**
 Trans(D,b) = **E**
 λ -clausura(Move(E,a)) = λ -clausura({3,8}) = **B**
 Trans(E,a) = **B**
 λ -clausura(Move(E,b)) = λ -clausura({5}) = **C**
 Trans(E,b) = **C**



2.3.7 Autómata Finito Determinista (AFD)

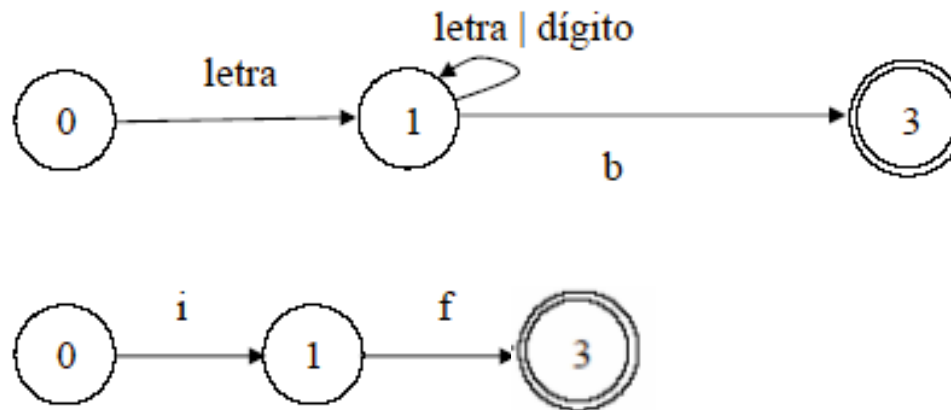
IMPLEMENTACIÓN DEL ANALIZADOR DE LÉXICO

Autómata reconocedor de patrones: AFD

Autómata reconocedor de palabras: AFD + Tabla de palabras reservadas

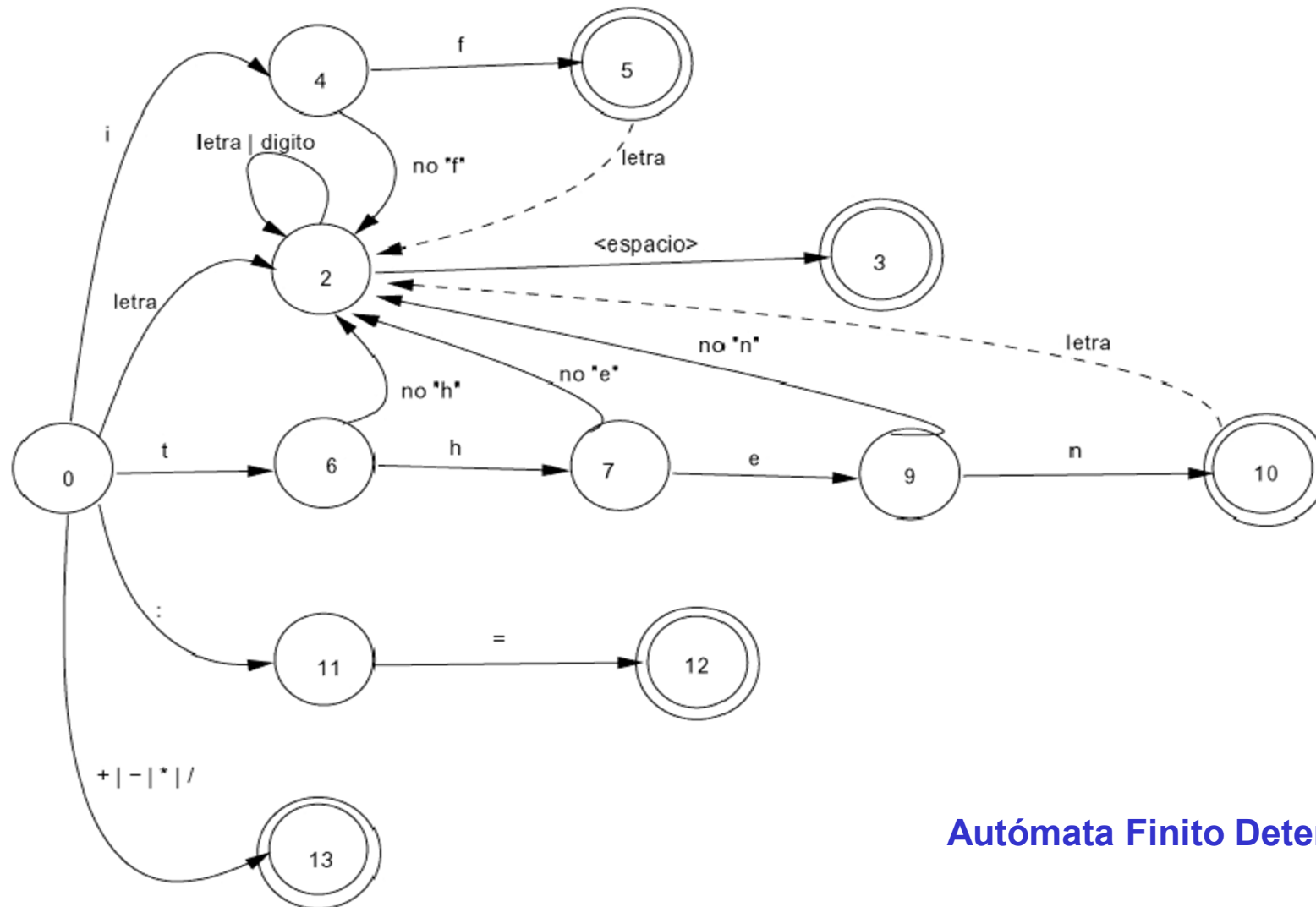
2.3.7 Autómata Finito Determinista (AFD)

PATRONES: Cada patrón es reconocido por un Autómata Finito Determinista



Problema de reconocer más de un patrón a la vez...

2.3.7 Autómata Finito Determinista (AFD)



Autómata Finito Determinista (AFD)

2.3.8 Gramática abstracta y tabla de tokens

Gramática abstracta: Gramática resultante de considerar los tokens como símbolos terminales y eliminar aquellas producciones en las que derivan los tokens.

Tabla de símbolos: Lugar de almacenamiento temporal (durante la fase de compilación y, en lenguajes orientados a objetos, durante la ejecución) debido a la necesidad de identificar los lexemas en la ocurrencia de cada token durante las fases posteriores de traducción.

Tabla de tokens: Tabla formada por tantas filas como tokens se hayan identificado. Las columnas tienen la siguiente información: Token, Código, Atributos y Patrón (expresión regular)

2.3.8 Gramática abstracta y tabla de tokens

Ejemplo 2.3: Dada la siguiente gramática, determinar el conjunto de tokens con el máximo nivel de abstracción para la construcción de un traductor.

$$\begin{array}{ll}
 P & \rightarrow S \mid P S \\
 S & \rightarrow \text{repeat exp } S; \\
 & \quad | \text{while exp } S; \\
 & \quad | \text{if exp } S; \\
 & \quad | \text{id = exp}; \\
 \text{exp} & \rightarrow \text{exp} * \text{exp} \\
 & \quad | \text{exp} / \text{exp} \\
 & \quad | \text{exp} + \text{exp} \\
 & \quad | + \text{exp} \\
 & \quad | * \text{exp} \\
 & \quad | (\text{exp}) \\
 & \quad | \text{constante} \\
 & \quad | \text{id} \\
 \text{constante} & \rightarrow \text{constante digito} \mid \text{digito} \\
 \text{digito} & \rightarrow [0 - 9] \\
 \text{id} & \rightarrow \text{id letra} \mid \text{letra} \\
 \text{letra} & \rightarrow [a - z]
 \end{array}$$

1. Encontrar todas las palabras/símbolos/signos de la gramática descritos por medio de su expresión regular.
2. Agruparlos atendiendo a la misión sintáctica, es decir, que desempeñen el mismo papel a nivel sintáctico.
3. Repetir el paso anterior hasta alcanzar la máxima abstracción, es decir, hasta que no sea posible realizar más agrupaciones.

Token	Código	Atributos	Patrón/Expresión regular
-------	--------	-----------	--------------------------

2.3.8 Gramática abstracta y tabla de tokens

Aplicando el punto 1, tenemos los siguientes elementos que podrían constituir tokens: *constante*, *digito*, *id* y *letra*.

$$\begin{array}{lcl} P & \rightarrow & S \mid P S \\ S & \rightarrow & \text{repeat exp } S; \\ & & \text{while exp } S; \\ & & \text{if exp } S; \\ & & \text{id = exp;} \\ \text{exp} & \rightarrow & \text{exp * exp} \\ & & \text{exp / exp} \\ & & \text{exp + exp} \\ & & + \text{exp} \\ & & * \text{exp} \\ & & (\text{exp}) \\ & & \text{constante} \\ & & \text{id} \end{array}$$

<i>constante</i>	$\rightarrow$	<i>constante digito</i> <i>digito</i>
<i>digito</i>	$\rightarrow$	[0 – 9]
<i>id</i>	$\rightarrow$	<i>id letra</i> <i>letra</i>
<i>letra</i>	$\rightarrow$	[a – z]

Dado que *digito* está incluido dentro de la especificación de *constante*, hace que *dígito* pueda eliminarse.

De igual modo, sucede con *id* y *letra* respectivamente.

Token	Código	Atributos	Patrón/Expresión regular
CONSTANTE			([0-9]) ⁺
ID			([a-z]) ⁺

2.3.8 Gramática abstracta y tabla de tokens

Aplicando los puntos 2 y 3, podemos extraer las siguientes palabras, signos y operadores y agruparlos atendiendo a la misión sintáctica:

$$\begin{array}{lcl}
 P & \rightarrow & S \mid P S \\
 S & \rightarrow & \text{repeat } \text{exp } S; \\
 & & \text{while } \text{exp } S; \\
 & & \text{if } \text{exp } S; \\
 & & \text{id} \equiv \text{exp}; \\
 \text{exp} & \rightarrow & \text{exp } * \text{exp} \\
 & & \text{exp} / \text{exp} \\
 & & \text{exp} + \text{exp} \\
 & & + \text{exp} \\
 & & * \text{exp} \\
 & & (\text{exp}) \\
 & & \text{constante} \\
 & & \text{id}
 \end{array}$$

De abajo hacia arriba, podemos observar los siguientes:

{ (,) , * , + , / , = , ; , if , while , repeat }

Según se mostraba en el Ejemplo 2.1, if, while y repeat constituyen un **token** dado que cumplen la misma misión sintáctica.

En cuanto a los operadores, hay dos tipos: *unarios/binarios* y *binarios exclusivos*. En cuanto a los primeros tenemos el * y el + que cumplen la misma misión sintáctica. Se agrupan.

Token	Código	Atributos	Patrón/Expresión regular
CONDCICLO	257	0: repeat 1: while 2: if	("repeat" "while" "if")
PYC	258		";"
ASIGN	259		"="
MULSUM	260	0: *, 1: +	("*" "+")
DIV	261		"/"
PARIZQ	262		"{"
PARDCH	263		"}"
CONSTANTE	264		([0-9])+
ID	265	Lexema o dirección de T.S.	([a-z])+

2.3.8 Gramática abstracta y tabla de tokens

La gramática resultante ha sido notablemente reducida con respecto a la original, tal y como se muestra a continuación:

```

P → S | P S
S → repeat exp S;
   | while exp S;
   | if exp S;
   | id = exp;
exp → exp * exp
   | exp / exp
   | exp + exp
   | + exp
   | * exp
   | (exp)
   | constante
   | id
constante → constante digito | digito
digito → [0 - 9]
id → id letra | letra
letra → [a - z]
```

```

P → S | P S
S → CONDCICLO exp S PYC
   | ID ASIGN exp PYC
exp → exp MULSUM exp
   | exp DIV exp
   | MULSUM exp
   | PARIZQ exp PARDCH
   | CONSTANTE
   | ID
```

2.4 Tratamiento de errores

Un traductor debe adoptar alguna estrategia para detectar, informar y recuperarse para seguir analizando hasta el final.

Las respuestas ante el error pueden ser:

- **Inaceptables:** Provocadas por fallos del traductor, entrada en lazos infinitos, producir resultados erróneos, y detectar sólo el primer error y detenerse.
- **Aceptables:** Informar de todos los errores y, aunque más complejo, reparar errores de forma automática.

2.4 Tratamiento de errores

El comportamiento de un analizador de léxico es el de un Autómata Finito o “scanner”.

Detección del error

El analizador de léxico detecta un error cuando no existe transición desde el estado que se encuentra para el símbolo de la entrada. El símbolo en la entrada no es el esperado.

Estrategias que se pueden adoptar

- **Pasar al estado inicial** ignorando los símbolos previos del lexema, iniciando de nuevo el proceso de identificación de los tokens.
- **Proceso de sincronización**, también conocido como **modo pánico (*panic*)**. Consiste en ignorar los símbolos de la entrada no esperados hasta que aparezca un símbolo esperado. Esta estrategia es buena cuando aparecen símbolos extraños de forma extra.
- **Reparar los errores** mediante transformación de los lexemas (borrar caracteres extraños, insertar caracteres que falten, intercambio de caracteres y sustitución de caracteres).

Tema 2 | Análisis de Léxico

Contenidos

2.1 Descripción funcional.

2.2 Conceptos de token, lexema y patrón.

2.3 Fundamentos: álgebra de lenguajes.

2.3.1 Alfabeto y lenguaje.

2.3.2 Operaciones con lenguajes.

2.3.3 Especificación de tokens y patrones.

2.3.4 Propiedades y notación taquigráfica de las expresiones regulares.

2.3.5 Autómata Finito no Determinista (AFN) asociado a una expresión regular.

2.3.6 AFN reconocedor de patrones.

2.3.7 Autómata Finito Determinista (AFD).

2.3.8 Gramática abstracta y tabla de tokens.

2.4 Tratamiento de errores.

2.4.1 Estrategias de recuperación.

2.4.2 Reparación de errores.

2.5 LEX (generador de analizadores de léxico).

2.5.1 Introducción.

2.5.2 Especificación LEX.

2.5.3 Consideraciones finales.

2.5.4 Ejemplos LEX.

Bibliografía básica

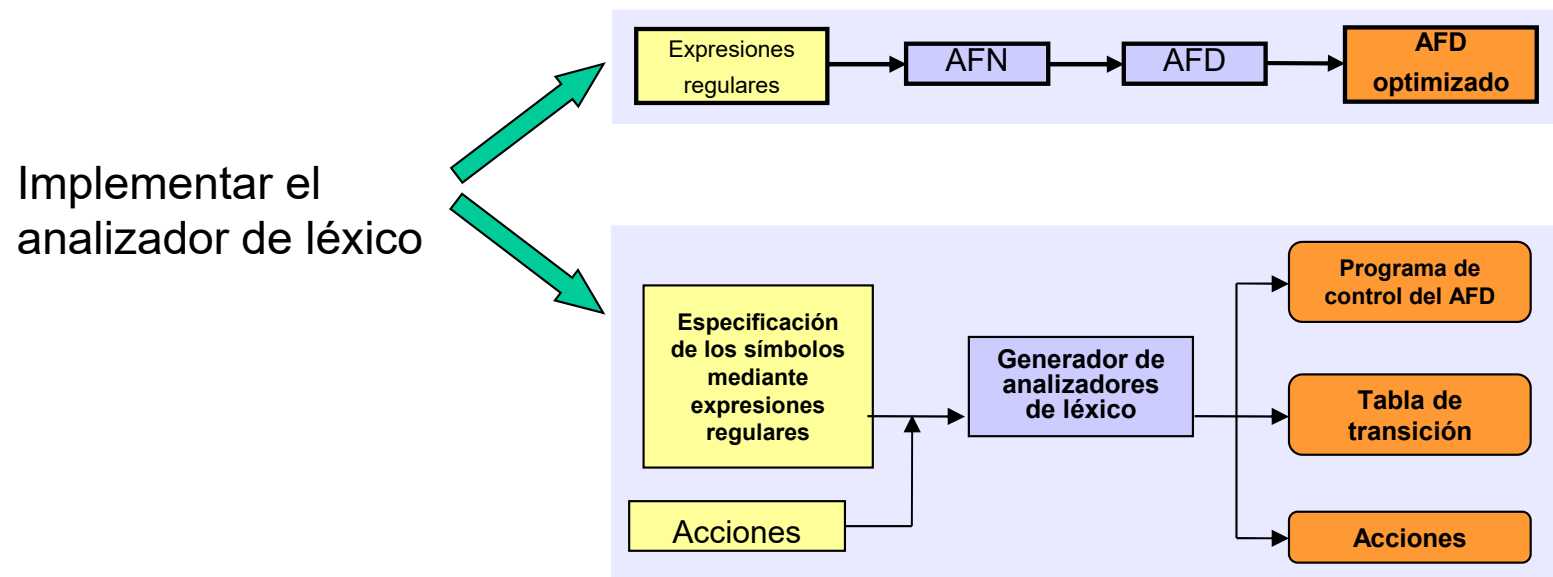
- [Aho90] Alfred V. Aho, Ravi Sethi, Jeffrey D. Ullman
Compiladores. Principios, técnicas y herramientas. Addison-Wesley Iberoamericana 1990.
- [Broo93] J.G. Brookshear
Teoría de la Computación. Lenguajes formales, autómatas y complejidad. Addison-Wesley Iberoamericana 1993

- [Levi92] J.R. Levine, T. Manson, D. Brown
Lex & Yacc
O'Reilly & Associates, Inc. 1992.
- [Benn90] J.P. Bennet
Introduction to compiling techniques: A first course using ANSI C, LEX y YACC
McGraw-Hill. 1990.

26/09/2026

2.5 LEX (Generador de analizadores de léxico)

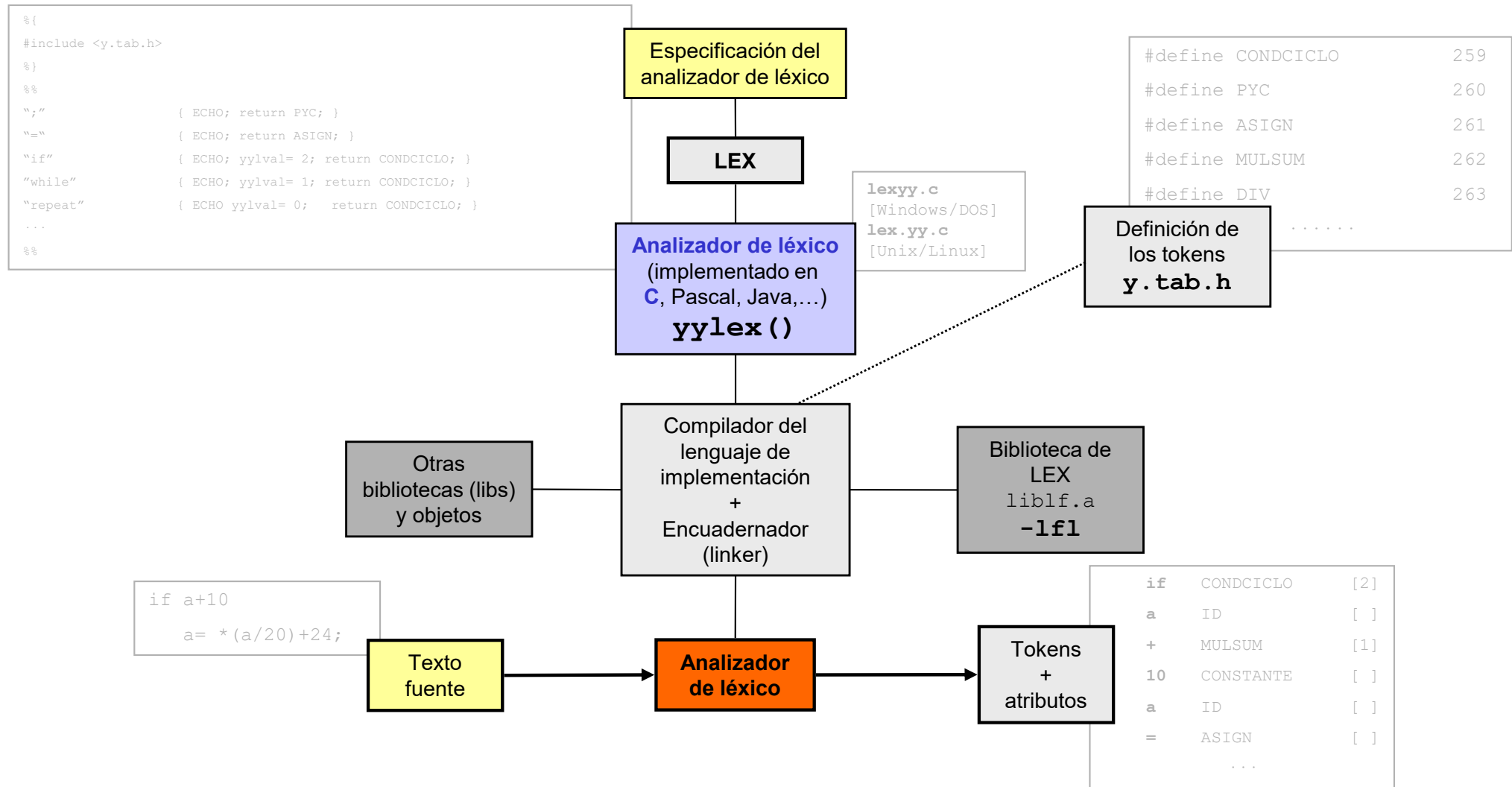
2.5.1 Introducción



Generador de analizadores de léxico: **LEX**

- Construir **analizadores de léxico** como programas independientes.
- Construir la rutina léxica que utiliza el **YACC** para generar **analizadores sintácticos**.
- Obtener programas que realicen análisis, estadísticas o cualquier tipo de **transformación a nivel léxico** en un texto fuente.

2.5.2 Especificación LEX



2.5.2 Especificación LEX

```
{Definiciones}
```

```
%%
```

```
{Reglas}
```

```
%%
```

```
{Procedimientos del usuario}
```

El mínimo programa LEX es:

```
%%
```

Su acción es la de copiar el fichero de entrada en uno de salida.

2.5.2 Especificación LEX: Definiciones

Pueden contener:

- De forma opcional, **código C** con el siguiente formato:

```
%{  
                                /* código C */  
%}
```

- **Definición** de un nombre mediante una **expresión regular** con la forma:

Nombre	expresión_regular
--------	-------------------

Ejemplo:

letra	[a-zA-Z]
digito	[0-9]

NOTA: Su posterior uso requerirá que vaya el nombre entre llaves, es decir, se tendrían que usar del siguiente modo: **{letra}** ó **{dígito}**

2.5.2 Especificación LEX: Reglas

Esta parte de la especificación **LEX** contiene los patrones que representan todos los símbolos a reconocer por el analizador de léxico, así como las acciones a realizar cuando son identificados.

Patrón (expresión regular)

{ Acción }

Ejemplo:

letra [a-zA-Z_]

digito [0-9]

%%

...

{letra}({digito} {letra})*	{ return ID; }
----------------------------	----------------

...

%%

2.5.2 Especificación LEX: Operadores

Operadores en <i>lex</i>	
*	Especifica la repetición de 0 o más veces (ejemplo a^*).
+	Especifica la repetición de 1 o más veces (ejemplo a^+).
()	Útiles para agrupar sub-expresiones.
	Especifica alternativa (ejemplo $a(bc de)$).
?	Especifica la opcionalidad del carácter precedente (ejemplo $ab?c$ reconoce ac o abc).
[]	Define una clase de caracteres, ejemplo $[xy]$. Representa al carácter x o al carácter y .
[m-n]	El operador — entre corchetes define un rango de caracteres entre m y n , según la tabla ascii. Fuera de los corchetes carece de significado, ejemplo: $[x - z]$ representa al carácter x , al carácter y o al carácter z .
[^x]	Indica cualquier carácter menos el indicado o indicados entre corchetes, ejemplo: $[^abc]$ representa cualquier carácter excepto a, b o c .
[\\]	Indica una constante de tipo carácter siguiendo la notación del lenguaje C como rango de caracteres imprimibles.
^	Especifica comienzo de línea.
\$	Especifica fin de línea.

2.5.2 Especificación LEX: Operadores

Operadores en <i>lex</i>	
.	Especifica cualquier patrón no especificado
"x"	Especifica que el carácter entre comillas no es un operador de Lex. (ejemplo "\$"no nos indica fin de línea).
\x	Especifica que el carácter que le sigue <i>x</i> no es un operador de Lex (igual que el caso anterior).
{ }	Especifica un patrón definido anteriormente, ejemplo: – digito [0 – 9]. – letra [<i>a</i> – <i>z</i>, <i>A</i> – <i>Z</i>]. – identificador6 {<i>letra</i>}({<i>letra</i>} {<i>digito</i>})[*].
x{ <i>m</i> , <i>n</i> }	Especifica que <i>x</i> aparece desde <i>m</i> a <i>n</i> veces, ejemplo: identificador6 { <i>letra</i> }({ <i>letra</i> } { <i>digito</i> }) [*] {0,5}.
x/ <i>y</i>	Reconoce un carácter cuando es seguido por un segundo caracter. En el ejemplo, solo reconoce <i>x</i> si va seguido de <i>y</i> .
\n	Indica fin de línea.
\t	Indica una tabulación.
\\	Indica el símbolo \$.
\b	Indica un espacio en blanco.

2.5.2 Especificación LEX: Operadores

Ejemplos: Expresiones regulares según la notación de LEX

[a-z] Representa cualquiera de las letras minúsculas de la **a** a la **z**

[-a-z] Representa cualquiera de las letras minúsculas de la **a** a la **z** y el signo **-**

(ab | cd+)? (ef)* Cadenas válidas: **abef**, **cdf**, **cdddd**, **efefef**

2.5.3 Consideraciones finales: Acciones

Las acciones representan la consecuencia de que el analizador de léxico haya reconocido un determinado lexema asociado a un patrón. El formato de las acciones es el siguiente:

<Acción>	::=	<Acción_Simple> "{" <Acción_Compuesta> "}"
<Acción_Simple>	::=	";" <Acción_Especial> ";" <sentencia_C> ";" " "
<Acción_Compuesta>	::=	<Acción_Simple> { <Acción_Simple> }
<Acción_Especial>	::=	ECHO REJECT BEGIN

Acciones por defecto

- Para los caracteres **reconocidos** → No realiza ninguna acción.
- Para los caracteres **no reconocidos** → Copiarlos en el fichero de salida (stdout).

Acciones ; y |

- **;** Representa la **acción por defecto** para los caracteres reconocidos.
- **|** Indica que la acción asociada al patrón es la **misma** que la del **siguiente patrón**.

Acciones especiales

- **ECHO**: Copia la secuencia de caracteres reconocidos en el fichero de salida.
- **REJECT**: Se usa para las reglas ambiguas.
- **BEGIN**: Útil cuando se desea tener construcciones sensibles al contexto por la izquierda.

2.5.3 Consideraciones finales: Variables y funciones LEX

Variables y funciones <i>lex</i>	
<code>char *yytex</code>	Variable que contiene el lexema reconocido. Al ser una cadena de caracteres, ésta finaliza con el carácter <code>\0</code> .
<code>int yyleng</code>	Número de caracteres reconocidos.
<code>int yylineno</code>	Línea actual del fichero de entrada.
<code>FILE *yyin</code>	Fichero de entrada, por defecto es stdin .
<code>FILE *yyout</code>	Fichero de salida, por defecto es stdout .
<code>void yymore()</code>	El siguiente token reconocido es añadido al actual en <code>yytex</code> , en lugar de reemplazarlo.
<code>int yywrap()</code>	Función llamada por el analizador de léxico cuando llega a fin de fichero y que, por defecto, devuelve el valor 1.
<code>int yyless(int n)</code>	Devuelve los <code>n</code> primeros caracteres del token actual al buffer de entrada para que vuelvan a ser analizados y así reajustar <code>yytex</code> y <code>yyleng</code> convenientemente.
<code>int input(int *c)</code>	Lee el siguiente carácter de la entrada.
<code>void unput(int c)</code>	Devuelve el carácter <code>c</code> a la entrada.
<code>void output(int c)</code>	Escribe el carácter <code>c</code> en la salida.

2.5.3 Consideraciones finales: Reglas ambiguas (acción REJECT)

LEX permite **especificaciones ambiguas**, es decir, especificaciones donde dos o más patrones se pueden corresponder con una misma secuencia de caracteres de entrada. En este caso la actuación del **analizador de léxico** es la siguiente:

1. Elige la **secuencia de caracteres más larga** que corresponde con un patrón.
2. A igualdad de caracteres **opta por el primer patrón**, es decir, el definido en primer lugar.

REJECT significa ir a la siguiente alternativa (patrón) y reexplorar el texto de entrada.

2.5.3 Consideraciones finales

Condiciones START (sensibilidad al contexto izquierdo)

Permiten **activar** o **desactivar** **patrones** dependiendo del **entorno** en el que se encuentren (**contexto**).

- Para definir los nombres de las condiciones, situaremos en la parte de las definiciones:

```
%START nombre_de_condicion_1 ... nombre_de_condicion_N
```

- La activación/desactivación de una condición se realizará mediante:
 - **BEGIN** (**nombre_condición**): Activación.
 - **BEGIN** 0: Desactivación de las condiciones.
- En las reglas se hará referencia a una condición de la siguiente forma:

```
<nombre_de_condición>      Patrón
```

Condiciones START exclusivas

En ellas cuando se activa una condición se desactivan todos los patrones que no comiencen por el indicativo **<nombre_de_condición>**. El formato solo difiere de las anteriores en la propia definición, es decir:

```
%X nombre_de_condicion_1 ... nombre_de_condicion_N
```

2.5.4 Ejemplos LEX

Ejemplo 2.1

```
%%  
[a-z]+    printf ("%s", yytext);  
%%  
  
main ()  
{  
    yylex() ;  
}
```

- Ejemplo 2.1 = Ejemplo 2.4.
- Ejemplo 2.2:
 - **Lexemas válidos:** no hace nada.
 - **Lexemas no válidos:** los copia en el archivo de salida.
- Ejemplo 2.3:
 - **Lexemas válidos:** los copia en el archivo de salida.
 - **Lexemas no válidos:** no hace nada.

Ejemplo 2.2

```
%%  
[a-z]+    ;  
%%  
  
main ()  
{  
    yylex() ;  
}
```

Ejemplo 2.3

```
%%  
[a-z]+    ECHO;  
.  
%%  
  
main ()  
{  
    yylex() ;  
}
```

Ejemplo 2.4

```
%%  
[a-z]+    ECHO;  
%%  
  
main ()  
{  
    yylex() ;  
}
```

2.5.4 Ejemplos LEX

Ejemplo 2.8: Contador de palabras, caracteres y líneas de un archivo.

```
%{
static unsigned num_caracteres = 0;      /* # de caracteres */
static unsigned num_palabras = 0;       /* # de palabras  */
static unsigned num_lineas= 0;          /* # de lineas    */
}%

%%

\n      num_caracteres += 2, ++num_lineas;      /* El límite de línea en MS-DOS es CR LF */
[^ \t\n]+ ++num_palabras, num_caracteres+= yyleng;
.       ++num_caracteres ;

%%

main ()
{
    yylex ();
    printf ("%d\t%d\t%d\n", num_caracteres, num_palabras, num_lineas) ;
    exit (0);
}
```

2.5.4 Ejemplos LEX

Ejemplo 2.9: Especificación ambigua. A la derecha se muestra un ejemplo de ejecución.

```
%%  
end          { ECHO; printf (" regla 1\n");  
endif        { ECHO; printf (" regla 2\n");  
a[cd]+       { ECHO; printf (" regla 3\n");  
a[bc]+       { ECHO; printf (" regla 4\n");  
%%  
  
main ()  
{  
    yylex() ;  
}
```

```
end  
end regla 1  
  
endif  
endif regla 2  
  
acddd  
acddd regla 3  
  
acccc  
acccc regla 3  
  
abbbb  
abbbb regla 4
```

2.5.4 Ejemplos LEX

Ejemplo 2.10

```
%{  
int e, a ;  
%}  
  
%%  
  
el          e++ ;  
ella       a++ ;  
\n          |  
.  
;  
  
%%  
  
main ()  
{  
    yylex() ;  
    printf ("%d, %d\n", e, a);  
}
```

Ejemplo 2.11

```
%{  
int e, a ;  
%}  
  
%%  
  
el          { e++ ; REJECT }  
ella       { a++ ; REJECT }  
\n          |  
.  
;  
  
%%  
  
main ()  
{  
    yylex() ;  
    printf ("%d, %d\n", e, a);  
}
```

2.5.4 Ejemplos LEX

Ejemplo 2.12

```
%%  
  
a[cd]+ { ECHO; printf ("...regla 1\n"); REJECT }  
a[cb]+ { ECHO; printf ("...regla 2\n"); REJECT }  
\n      |  
.  
;  
  
%%  
  
main ()  
{  
    yylex() ;  
}
```

acddd

```
acddd ...regla 1  
acdd ...regla 1  
acd ...regla 1  
ac ...regla 1  
ac ...regla 2
```

acbbb

```
acbbb ...regla 2  
acbb ...regla 2  
acb ...regla 2  
ac ...regla 1  
ac ...regla 2
```

Ejemplo 2.13

```
%%  
  
a[cd]+ { ECHO; printf ("...regla 1\n"); REJECT }  
a[cb]+ { ECHO; printf ("...regla 2\n"); }  
\n      |  
.  
;  
  
%%  
  
main ()  
{  
    yylex() ;  
}
```

acddd

```
acddd ...regla 1  
acdd ...regla 1  
acd ...regla 1  
ac ...regla 1  
ac ...regla 2
```

acbbb

```
acbbb ...regla 2
```

2.5.4 Ejemplos LEX

Ejemplo 2.14

```
%{  
    int i;  
}%  
  
%%  
  
^a      {i='a'; ECHO; }  
^b      {i='b'; ECHO; }  
^c      {i='c'; ECHO; }  
\n      {i=0  ; ECHO; }  
  
letra   { switch (i)  
        { case 'a' : printf (" primera\n"); break ;  
          case 'b' : printf (" segunda\n"); break ;  
          case 'c' : printf (" tercera\n"); break ;  
          default  : ECHO ; break ;  
        }  
      }  
  
%%  
  
main ()  
{  
    yylex () ;  
}
```

acelera que nos vamos letra
acelera que nos vamos primera

buscando desesperadamente letra
buscando desesperadamente segunda

cada vez se ve mejor letra
cada vez se ve mejor tercera

2.5.4 Ejemplos LEX

Ejemplo 2.15: Condiciones START

```
%START A B C

%%

^a      { BEGIN A; ECHO; }
^b      { BEGIN B; ECHO; }
^c      { BEGIN C; ECHO; }
\n      { BEGIN 0; ECHO; }

<A>letra      printf (" primera\n");
<B>letra      printf (" segunda\n");
<C>letra      printf (" tercera\n");

%%

main ()
{
    yylex () ;
};
```

```
acelera que nos vamos letra
acelera que nos vamos  primera

buscando desesperadamente letra
buscando desesperadamente  segunda

cada vez se ve mejor letra
cada vez se ve mejor  tercera
```

2.5.4 Ejemplos LEX

Ejemplo 2.16: Condiciones START exclusivas

```
%X A B C

%%

^a      { BEGIN A; ECHO; }
^b      { BEGIN B; ECHO; }
^c      { BEGIN C; ECHO; }
\n      { BEGIN 0; ECHO; }
numero  { printf ("##"); }

<A>letra      printf (" primera\n");
<B>letra      printf (" segunda\n");
<C>letra      printf (" tercera\n");

%%

main ()
{
    yylex () ;
};
```

```
acelera que nos vamos letra
acelera que nos vamos  primera

buscando desesperadamente letra
buscando desesperadamente  primera

cada vez se ve mejor letra
cada vez se ve mejor  primera

buscando desesperadamente letra
buscando desesperadamente  segunda

cada vez se ve mejor letra
cada vez se ve mejor  segunda

acelera que nos vamos letra
acelera que nos vamos  segunda

dadas las circunstancias numero
dadas las circunstancias ##

cuando comencemos así letra
cuando comencemos así  tercera

dadas las circunstancias numero
dadas las circunstancias numero
```

2.5.4 Ejemplos LEX

Ejemplo 2.17: Analizador léxico para un lenguaje simple

```
%{
#include <stdlib.h>
#include <string.h>
#include "tabla.h"

int linea_actual=0 ;
%}
letra      [a-zA-Z]
digito     [0-9]
alphanum   [a-zA-Z_0-9]
blanco     [ \t]
otros      .

%%
"("        return PIZ;
")"        return PDE;
"+"        return SUM;
"-"        return SUB;
"*"        return MUL;
"/"        return DIV;
":="       return ASI;
{letra}{alphanum}*
{return ID;
{digito}+   {return ENT;
{blanco}+   {return BL;
\n          ++linea_actual ;
{otros}     printf ("\n(Linea %d) Error léxico: token %s\n", yylineno, yytext);
%%
main ()
{
    int val;
    val= yylex() ;
    while (val != 0)
    {
        printf (" %d\n", val);
        val= yylex() ;
    }
    exit (1);
}
```

```
/* tabla.h */

#define SUM  257
#define MUL  258
#define DIV  259
#define SUB  260
#define ENT  261
#define PDE  262
#define PIZ  263
#define ASI  264
#define ID   265
#define BL   266
```

2.5.4 Ejemplos LEX

Ejemplo 2.18: Dado el lenguaje descrito por el siguiente conjunto de producciones

Programa → Expresion ; Programa
Expresion → Expresion + Expresion
 | Expresion - Expresion
 | Expresion * Expresion
 | Expresion / Expresion
 | (Expresion)
 | - Expresion
 | Numero
Numero → Numero Digito | Digito
Digito → 0 | 1 | ... | 9

La **tabla de tokens** con máximo nivel de abstracción es:

Token	Número	Patrón
OPBIN	257	+ * /
OPUNA	258	-
PARIZQ	259	(
PARDCH	260	)
PCOMA	261	;
NUMERO	262	Digito{Digito}

<pre> %{ #include "tabla.h" }% numero [0-9]+ otros . %% "+" return OPBIN ; "*" return OPBIN ; "/" return OPBIN ; "-" return OPUNA ; "(" return PARIZQ ; ")" return PARDCH ; ";" return PCOMA ; [\t\n] ; {numero} return NUMERO ; {otros} printf ("\n(Linea %d) Error léxico: token %s\n", yylineno, yytext); %% main () { int val; val= yylex() while (val != 0) printf ("%d \n", val) ; val= yylex() ; exit (0); } </pre>	<pre> #define OPBIN 257 #define OPUNA 258 #define PARIZQ 259 #define PARDCH 260 #define PCOMA 261 #define NUMERO 262 </pre>
--	---